Make StreetFighter II Champion Using Reinforcement Learning

2015410035 2017320144 2014190704 2015410059
김현승 강신환 유성동 이하빈

요 약

모든 상황이 정해져 있는 환경을 학습하고, 그 환경 안에서만 행동하는 것은 AI에게 있어 크게 어려운 일이 아니다. 학습을 통해 발생할 모든 경우의 수를 기억하고 그에 맞춰 대응하기만 하면 되기 때문이다. 그러나 이런 AI는 자신이 학습한 범위 밖의 상황을 맞이하였을 때 분명한 한계를 드러내게 된다. 그래서 우리는 자신이 모르던 상황에 대처할 수 있는 AI를 만들기 위해 rule-base의 게임들이 아닌, stochastic한 게임 환경에서 학습을 진행하였다. 이 프로젝트에서는 1990년대의 대표적인 격투게임인 Street Fighter II에서 모든 캐릭터를 이길 수 있는 Agent를 만드는 것을 목표로 하였고, 해당 목표를 이룰 수 있도록 여러가지 강화학습 기법을 통한 학습을 진행하였다.

1. 서 론

현실 세계에는 매초마다 수많은 사건들이 일어난다. 인공지능이 현실 세계로 나가려면 학습을 통해 알고 있지 않은 상황에서도 적절한 행동을 취할 수 있는 능력을 가져야 한다. 자동차의 자율 주행을 예로 들어보자. 자율 주행 기술은 많은 사람들이 주목하는 만큼 의구심을 품고 있는 사람들도 많은데, 이런 사람들은 주로 돌발 상황, 혹은 미지의 상황에 직면했을 때도 안전하게 대처를 할 수 있는가를 의심하곤 한다. 실제 도로 주행에는 셀 수도 없을 만큼 많은 요소들이 개입하고, 그로 인해 발생할 수 있는 경우의 수 또한 무수히 많다. 이런 환경에서 운전대를 잡고 있는 AI가 발생할 모든 상황을 암기하고, 그 기억에만 의존하여 대처하는 AI라면, 도로에 나간 지 5분도 안되어 명을 다할 것이다. 이런 인공지능은 현실 세계에서 절대 신뢰할 수 없다. 주어진 환경에 대해 모든 정보를 갖지 못하는 우리가 필요로 하는 인공지능은 자신이 모르고 있던 새로운 상황을 마주하여도 적절한 행동을 취할 수 있는 것이다.

강화학습은 Agent가 환경과의 상호작용을 통해 학습하면서 주어진 상황에 대해서 최적의 행동을 할 수 있도록 하는 알고리즘이다. 목표에 도달할 때까지 매시간마다 받는 보상을 최대로 하는 것을 목표로 한다. 강화학습의 학습을 위해서는 현재 상황, Agent의 행동, 다음 상황, 보상 등을 제공해주기 위한 환경이 필요하다. 그래서 강화 학습이 현실 세계로 적용되기엔 결정적인 한계가 있었다. 앞서 언급했듯이 현실을 구성하고 있는 요소들은 너무도 많기 때문에 그것을 모두 포함하는 환경을 만들기는 불가능하다. 만약 가능하다고 하더라도 그 많은 정보를 전부 처리하면서 학습을 하기에는 너무나도 많은 시간을 요한다. 게다가 Agent의 행동에 따른 보상을 현실에서 명확하게 정의하는 것 또한 너무도 어려운 일이었다.

이런 강화 학습의 한계를 극복하기 위해서 나온 방안이 강화 학습과 딥러닝의 결합, 딥강화학습이다. 상태에 대한 정보를 인간이 수동으로 주는 것 대신, 시각적 정보를 통해 Agent가 판단할 수 있도록 한 것이다. 2013년, 딥강화학습을 이용해 벽돌 깨기와 같은 단순한 고전 게임들, 통칭 Atari 게임들을 모두 정복하는 AI가 개발되었으며, 이후에도 슈퍼 마리오 등의 게임을 차례차례 공략해내는 AI가 개발되었다. 하지만 이것들 또한 현실에 적용이 될 수 있는가에 대해서는 물음표를 지울 수 없었다. 여태 공략된 게임들은 대부분이 극도로 단순한 게임이거나, 발생 가능한 상황들이 이미 정해져 있어 변하지 않는다는 공통점을 지니고 있기 때문이다. 그러기에 이들이 지속적인 학습의 결과 이 게임을 전부 외워서 공략을 한 것이라면, 이들 또한 현실에 적용되기는 힘들 것이라는 사실은 마찬가지기 때문이다.

우리는 non-deterministic한 환경을 가정하기 위해서 stochastic한 게임을 공략함으로써 한 발짝 더 현실에 다가간 AI를 개발하기로 하였다. 그래서 선택하게 된 게임이 Street Fighter II라는 격투 게임이다. 격투 게임의 특성상 자신과 상대의 지속적인 상호 작용을 통해 다음 행동을 결정해야 한다는 점에서 stochastic한 환경을 지니고 있었다. 또한 12명의 캐릭터를 모두 제압할 수 있게 된다면 모델의 generality도 보일 수 있을 것이다.

그러기 위해 여러가지 강화학습 기법 중에서 Deep Q Network (DQN)과 Advantage Actor Critic (A2C)를 재구현하고, Character Classification Layer을 추가하여 Agent의 학습을 진행하였다. 처음에는 한 캐릭터만을 공략하는 것을 시작으로 해서 12명의 캐릭터를 전부 제압할 수 있도록 하여 Agent의 generality를 확보했다.

2. 관련연구.

추가적으로 Street Fighter 환경과 같이 똑같아 보이는 상태라도 게임의 난이도가 더 어려울 경우 제대로 학습

하지 못하는 문제가 존재했다. 만약 에이전트 입장에서 문제가 너무 어려울 경우 학습시간이 길거나 제대로 수렴하지 못하는 문제를 해결하기 위해 Curriculum Learning이라는 학습 방식을 선택하기도 하였다 [4]. Curriculum Learning을 통해 쉬운 문제를 먼저 해결하고 난이도를 올려가며 어려운 문제를 해결하는 방식을 통해 성능을 높였다. 실제로 [5]에서는 이미지를 분류할 때 이미지들에 대한 난이도 및 주변과의 비슷함 정도를 측정한 뒤 가장 쉽고 정확하게 판단할 수 있는 이미지를 분류한 뒤 다시 앞의 과정을 반복하는 식인 Curriculum learning을 사용하여 성능을 향상시켰다.

3. 환 경

Gym Retro란 강화학습을 게임에 적용하기 위해 출시된 플랫폼이다. Atari, Sega 게임 등 1000가지가 넘는 게임을 강화학습을 위해 다양한 툴을 지원한다.

본 과제는 Gym Retro에 존재하는 게임 중 'StreetFighterIISpecialChampionEdition-Genesis'(이하 SF)를 사용하여 진행하였고, 학습하는 캐릭터의 이름은 'Ryu'로 state명은 'ryu'이다. 여기서, 언급한 state명이란 게임 환경을 불러오는 키 값을 말한다. 본 과제에서는 모든 state를 모두 이용하지 않고 'ryu'를 포함한 각각 다른 normal단계의 12개의 캐릭터를 이기는 것이 목표이다.

구현한 DQN, A2C의 성능확인을 위해 '퐁'을 사용하였다. 하지만, 에이전트가 선택할 수 있는 액션의 수가 '퐁'과는 다르게, 'SF'에서는 대폭 늘어났다. '퐁'에서는 단순히 retro에서 제공하는 3가지면 충분했지만 'SF'의 경우에는 존재하는 버튼 수가 늘어남과 동시에 대각선 점프, 스킬 등 여러 버튼을 함께 눌러주는 경우들을 모두 고려해야하기 때문에 액션을 정의하는 함수를 게임 내부의 Wrapper를 사용하여 커스터마이징 해주었다.

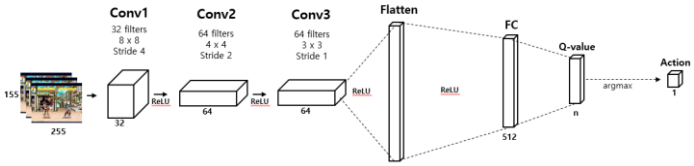
'퐁'의 경우 화면 구성이 간단하여서 resize를 해주어 화면을 4개까지 쌓아 학습을 진행할 수 있었지만, 'SF'의 경우에는 화면 구성이 복잡하여 resize하게 된다면 학습이 잘 이루어지지않는 문제가 있었다. 그래서 이를 해결하기 위해서 resize를 사용하지 않았고, 화면 내에서 학습과는 무관하지만 계속해서 변화하는 체력, 시간, 관중 들을 화면에서 잘라내어 사용하였다.

'퐁'에서 사용한 리워드는 오직 승리+1, 패배 -1이었다. 하지만, 'SF'에서는 기존 내장된 승리/패배, 게임이 종료되었을 때 남는 시간, 체력 감소에 따른 리워드들을 'scenario.json'파일을 통해 조절할 수 있었다. 위의 파일에서 Reward의 비율을 조정함으로써, 캐릭터의 성향(공격적, 방어적)을 정할 수도 있었다. 본 과제에서는 기본으로 제공하는 리워드를 조작하여 사용해보고, 게임 내에서 제공하는 'info'를 사용한 임의 Reward를 삽입하여 실험해보는 등 여러가지 방법을 시도해보았다. 여기서 언급한 'info'는 캐릭터들의 남은 체력, 남은 시간, 승리/패배 카운트를 제공한다.

4. 모 델

총 DQN, A2C 두가지 알고리즘을 사용하였고, 성능 향상을 위해 새로운 network에 추가하였다.

4.1 DQN(Deep Q Network)

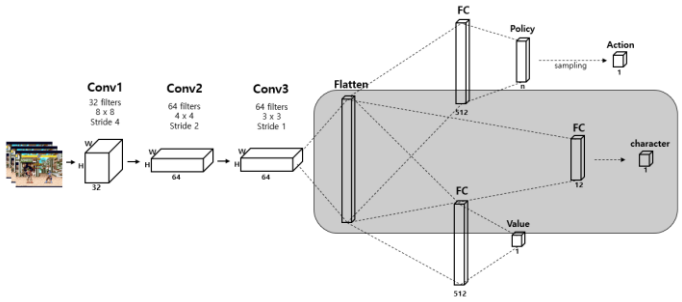


<그림 1. DQN 모델 구조>

DQN은 Neural Network를 이용하여 현재 시점에서 특정 행동(Action)을 했을 때 먼 미래까지 고려하여 얻을 수 있는 값(optimal action-value function(Q*))을 구한다. 결국, 최적의 Q*를 구함으로써 Agent가 어떤 input이 들어와도 최적으로 행동(action)을 결정할 수 있는 정책(Policy)을 찾게 되는 것이다.

이런 방식을 사용한 DQN은 게임화면과 점수만 인풋으로 받아서 기존의 성능을 넘거나, 사람보다 뛰어난 성능을 보이는 게임도 있었다. 또, DQN의 다른 장점은 동일한 하이퍼파라미터를 사용하면서 서로 다른 종류의 2600개의 게임에 적용할 만큼 general한 모델인 것이 증명되었다.

4.2 A2C(Advantage Actor Critic)



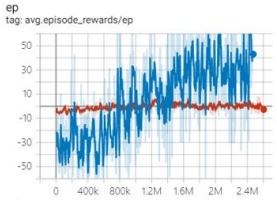
<그림 2. Character Classify A2C 모델 구조>

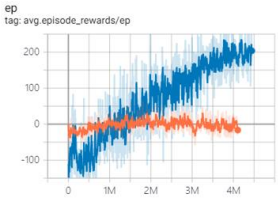
우리는 목표로 하는 Agent를 만드는데 DQN으로는 한계가 있다고 생각하였다. 그래서 찾은 다른 강화학습 방법론은 A2C이다. A2C는 DQN과 다르게 Q값을 최적화해서 policy를 구하는 것이 아닌 policy 자체를 최적화하는 방법론이다. Loss는 총 policy loss, entropy loss, value loss로 3가지가 있고 합쳐서 학습한다. 우리가 A2C를 task에 적용하게 된 이유는 Actor 여러 개가 한꺼번에 학습이 진행되기 때문이다. 하나의 Agent가 하나의 네트워크를 학습하는 DQN과 달리 A2C는 내가 지정한 개수의 Actor가 하나의 네트워크를 공유하면서 각각 학습한다. 여러 Actor가 있다는 것은 우리의 목표와 환경에 있어서 큰 장점으로 사용할 수 있다. 그리고 기존 모델이 환경 속 12가지의 캐릭터를 구별하는데 한계가 있어서 캐릭터를 구별해주기 위한 character classification network를 추가해주었다.

5. 실 험

본 실험은 앞서 언급한 DQN과 A2C로 진행되었다. 먼저, 한 가지의 캐릭터의 normal 난이도(바이슨)를 이기는 것을 첫 목표로 정하였다. 그 후, 충분한 리워드가 오른다면 모든 캐릭터를 이길 수 있는 general한 모델을 만들고자 시도하였다.

5.1 DQN





<그래프 1. 리워드 균등배분(파란선), 리워드 불균등배분(빨간선)>

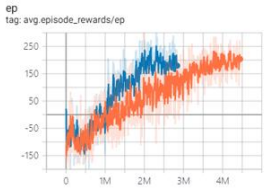
<그래프 2. 원본 이미지 제공(파란선), resize이미지 제공(빨간선)>

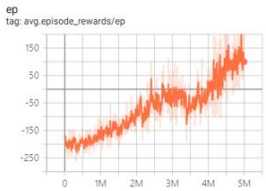
- 리워드의 배분 차이 실험

<그래프 1>에서 볼 수 있듯, 결과적으로, 0.5씩 균등하게 줬을 때가 리워드가 월등히 높은 것을 확인할 수 있다.

- 이미지 해상도에 따른 차이 실험

'퐁'에서는 84x84의 이미지를 4장 쌓아서 input으로 사용하였지만, 화면 구성이 복잡해짐에 따라 학습이 잘 이루어지지않았다. <그래프 2>에서 볼 수 있듯, resize를 거치지 않은 원본의 이미지를 제공하는 것이 학습이 resize가 이루어진 학습보다 더 잘 됨을 볼 수 있었다.





<그래프 3. 고정 LR(파란선), LR scheduler(빨간선)>

<그래프 4. 바이슨 첫 클리어>

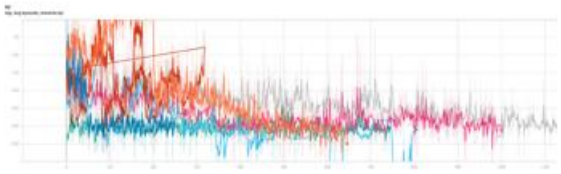
- Learning Rate에 따른 차이 실험

Learning rate(이하 LR)을 고정으로 쓰는 실험에서 특정 양수 값에 수렴하기는 한다. 하지만, 너무 느린 것 같아서 LR를 scheduler형식으로 바꾸어 보았다. 초기 LR은 0.00025에서 시작하여 선형적으로 0.0005까지 감소시켜주었다. <그래프 3>에서 볼 수 있듯, 수렴속도가 훨씬 빨라짐을 볼 수 있었다.

- RM(바이슨)-Normal 첫 클리어

앞선 실험에서 RM-EASY를 이긴 후, <그래프 4>에서 볼 수 있듯 RM-Normal는 모델을 생성하는데 성공하였다. 하지만, 한 가지 캐릭터에 대해 계속 학습을 해줌으로써 인해 모든 상황에서 같은 행동을 하는 Rule-Base처럼 되어버리는 것을 관찰할 수 있었다. 심지어 RM-Normal은 이길 수 있었지만, RM-Easy를 이기지 못하였다. 따라서, general하지 못하다고 볼 수 있다.

- 이후 실험

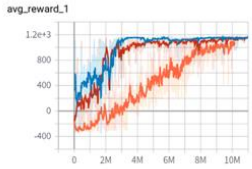


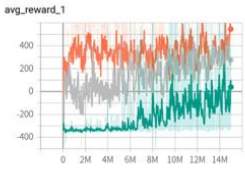
<그래프 5. 그 후의 실험들>

이후에 하이퍼 파라미터들을 바꿔보며 다양한 실험들을 해보았지만, DQN은 한가지 캐릭터는 CLEAR할 수 있었지만 다양한 캐릭터에 대해서는 리워드가 잘 오르지 않았다.

<그래프 5>에서 잠시 위로 오르는 선을 볼 수 있는데 이는 Curriculum Learning을 위해 한 캐릭터만 약 200만스텝을 학습한 실험이다. 일정 스텝을 학습한 후, 모든 캐릭터에 대해 학습을 시작하자 리워드가 떨어지는 모습을 볼 수 있었다.

5.2 A2C





그래프 6. easy 12 (주황선), easy 6, normal 6(빨간선), normal 12(파란선)

그래프 7. general agent 학습 1~1500만회(초록선), 1500만~3000만회(회색선), 3000만회~4500만회(주황선)

A2C도 DQN과 마찬가지로 한가지 캐릭터를 이기는 것부터 시작하였다. Easy단계의 바이슨을 이기는 것은 매우 간단하였다. DQN은 normal 단계의 바이슨을 완벽히 클리어하지 못했었다. A2C에서는 여러 Actor가 있다는 장점을 사용해서 curriculum learning을 사용해서 Normal 단계의 바이슨을 완벽히 클리어 할 수 있었다. curriculum learning이란 쉬운 task부터 시작해서 점점 난이도를 높여가면서 training 하는 방법론이다. A2C의 actor들을 easy와 normal로 원하는 만큼 지정해주었다. Easy와 Normal의 비율을 순서대로 12:0, 6:6, 0:12로 학습을 이어서 진행하였다. 그 결과 <그래프 6>에서 최고 reward를 찍고 normal 바이슨 상대로 한번도 데미지를 입지 않고 완벽하게 이길수 있었다.

이제 한가지 캐릭터를 넘어서 모든 normal 캐릭터를 이길수 있는 General한 Agent를 만드는 것을 목표로 하였다. Street Fighter II에는 총 12가지 캐릭터가 있기 때문에 12개의 Actor에 한가지 캐릭터를 각각 배정하였다. 학습을 한 결과 Reward가 많이 상승하는 것을 확인하였다. 그러나 문제점이 한가지 존재하였다. 모든 캐릭터를 상대로 똑같은 policy를 학습해서 장거리 공격패턴을 갖는 달심에게 이기지 못하는 것을 발견할 수 있었다. 그래서 이 문제를 해결하기 위해 새로운 network를 만들어서 추가해주었다. Agent가 캐릭터들을 구분하고 각

기 다른 policy를 학습하게 하기 위해서 Character Classification Network를 추가하였다. 기존의 3가지 Loss에 Classification Loss를 추가하여 학습하였다. 결과는 <그래프7>에서 볼 수 있다. 캐릭터마다 다른 policy를 학습할 수 있었고 12가지 normal 캐릭터를 상대로 한번도 지지 않고 이길 수 있었다.

5.3 Other Method

현재 강화학습을 포함한 Neural Network 방식을 통해 우리가 원하는 가중치를 학습할 수 있는 이유는 역전파의 등장 덕분이다. 우리가 원하는 Neural Network는 Input을 넣으면 가중치를 통해 Output을 만들어 내며 이 방식을 순전파라고 하며, 이로 인해 나온 결과와 우리가 원하는 결과의 차이를 통해 가중치를 학습시키는 방식이 바로 역전파이다.

우리가 구현한 역전파 중에서 가장 주가 되는 부분은 바로 CNN의 역전파 부분이다. CNN은 기본적으로 이미지를 pixel 형태로 바꾼 행렬 꼴의 Input에 필터라는 것을 통과시켜 이미지의 특징을 잡아내는 구조로 진행된다.

x_{11}	x_{12}	x_{13}
x_{21}	x_{22}	x_{23}
x_{31}	x_{32}	x_{33}

표 1 Input

F_{11}	F_{12}
F_{21}	F_{22}

표 2 Filter

O_{11}	O_{12}
O_{21}	O_{22}

표 3 Output

CNN에서는 필터가 Input을 돌아다니면서 성분곱셈을 통해 Output을 내보내게 된다. 우리가 역전파를 통해 업데이트하려는 값은 바로 필터들의 성분이기 때문에 순전파을 통해 손실을 계산하고, 이 손실 줄이는 방향으로 성분들을 변화시켜주어야 한다. 결과에서 나온 에러를 미분을 통해 각 필터의 성분이 에러에 얼마나 영향을 끼치는지 조사하고 에러를 줄이는 방식으로 업데이트를 하면 된다. 수식으로 써보면 다음과 같다.

$$\frac{\partial E}{\partial F_{11}} = \frac{\partial E}{\partial O_{11}}X_{11} + \frac{\partial E}{\partial O_{12}}X_{12} + \frac{\partial E}{\partial O_{21}}X_{21} + \frac{\partial E}{\partial O_{22}}X_{22}$$

$$\frac{\partial E}{\partial F_{12}} = \frac{\partial E}{\partial O_{11}}X_{12} + \frac{\partial E}{\partial O_{12}}X_{13} + \frac{\partial E}{\partial O_{21}}X_{22} + \frac{\partial E}{\partial O_{22}}X_{23}$$

$$\frac{\partial E}{\partial F_{21}} = \frac{\partial E}{\partial O_{11}}X_{21} + \frac{\partial E}{\partial O_{12}}X_{22} + \frac{\partial E}{\partial O_{21}}X_{31} + \frac{\partial E}{\partial O_{22}}X_{32}$$

$$\frac{\partial E}{\partial F_{22}} = \frac{\partial E}{\partial O_{11}}X_{22} + \frac{\partial E}{\partial O_{12}}X_{23} + \frac{\partial E}{\partial O_{21}}X_{32} + \frac{\partial E}{\partial O_{22}}X_{33}$$

또한 손실을 줄이는 방식으로 우리는 경사하강법을 사용했다. 우리가 변화시키려는 변수로 이루어져 있는 손실함수를 변수에 대한 미분을 통해 기울기의 반대방향만큼 이동시켜 줌으로써 손실함수를 가장 작게하는 값을 찾아내는 방법이다. 이때 기울기의 반대방향으로 이동시키는 방식에도 여러가지가 있는데, 그 중에서도 우리가 사용한 방식은 RMSProp이라는 방식이다.

기본적으로 경사하강법을 사용할 시 기울기를 계산한 뒤 값을 얼마나 변화시킬건지에 대한 수치로 학습률이라는 것을 사용하는데, 이것은 우리가 변수를 학습시키는 데 있어 적절한 값을 찾아주는 것이 중요하다. RMSProp에서는 이 학습률의 값을 찾을 때 단순히 하나의 값으로 학습을 진행하기 보다는 각각의 변수들에 대한 과거에 반영된 기울기를 조사하여 변수별로 다른 학습률을 가지게 한다. 이런 방식을 사용하였을 때 뒤로 갈수록 학습의 속도가 현저하게 낮아진다는 점이 있었는데, RMSProp은 최근값의 제곱근으로 나누어 주어 이를 해결시켰다. RMSProp의 Pseudo code는 다음과 같다.

Algorithm 1: RMSProp Pseudo code

Input: Input: $\alpha(global\ learning\ rate), p(decay\ rate)$
Initialization : $\theta(parameter), r = 0(accumulation\ variables), \delta = very\ small\ value$
Sampling $x^{(1)}, x^{(2)}, ...x^{(n)}\ from\ Dataset\ corresponding\ target\ y^{(i)}$;
while do
  $g \leftarrow 1/m \nabla_\theta \Sigma_i L(f(x^{(i)};\theta), y^{(i)})$ *compute gradient*
  $r \leftarrow pr + (1-p)g^2$ *get accumulate squared gradient*
  $\Delta\theta = -\alpha/\sqrt{\delta + r} * g$ *parameter update*
  $\theta \leftarrow \theta + \Delta\theta$
end

6. 향후 연구

강화학습은 보상을 기반으로 학습하기 때문에 보상을 어떻게 정하는지에 따라 Agent의 정책을 조정할 수 있다. Agent의 체력이 깎이는 것을 penalty로 설정하였을 때 agent는 방어형으로 게임을 진행하였고, agent의 체력이 깎이는 것에 아무런 penalty를 주지 않았을 때 agent는 공격적으로 게임을 진행하였다. 그리고 두개를 적절히 섞어서 한 결과 가장 좋은 성능을 나타내는 agent를 만들 수 있었다. 하지만 DQN, A2C 알고리즘이 각광받았던 이유는 같은 하이퍼파라미터, 단순한 보상으로 모든 Atari game을 정복했기 때문이다. 그래서 보상을 직접 조정하는 것 보다 이겼을 때 1점 패배했을 때 -1점으로 설정 후 내재적인 보상을 agent가 직접 획득해 나가면서 학습하는 모델을 만드는 것이 정말 general 한 알고리즘이라고 할 수 있다. 선행 연구로 인간의 호기심을 본 따서 탐험을 할 수 있게 만든 ICM 모듈이 존재한다. 이렇게 agent가 다양한 경험을 하게 하는 것이 좀 더 general 한 agent를 학습할 수 있도록 만든다고 생각한다.

7. 역할 분장

1. 김현승(조장): DQN, A2C논문 재구현, A2C로 sf 학습, Curriculum Learning 적용
2. 강신환: DQN논문 재구현, DQN으로 sf 학습
3. 유성동: backpropagation, update algorithm 구현
4. 이하빈: backpropagation, update algorithm 구현

References

[1] Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., and Riedmiller, M. Playing atari with deep reinforcement learning. Advances in Neural Information Processing Systems (NIPS), 2013.

[2] Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., and Hassabis, S. L. . D. Human-level control through deep reinforcement learning. Nature, 2015.

[3] Mnih, Volodymyr, et al. "Asynchronous methods for deep reinforcement learning." International conference on machine learning. 2016.

[4] Bengio, Yoshua, et al. "Curriculum learning." Proceedings of the 26th annual international conference on machine learning. ACM, 2009.

[5] Jiang, Lu, et al. "Easy samples first: Self-paced reranking for zero-example multimedia search." Proceedings of the 22nd ACM international conference on Multimedia. ACM, 2014.